

# Proyecto Práctico. Trabajo en pareja con Git Colaborativo



# {"PrOgr4m4 tu futur0"};

Edición Flexible Back End Java

Contexto **01**

---

Base de datos **02**

---

Organización y arquitectura **03**

---

División de tareas **04**

---

Fase de integración - Reservas **05**

---

Flujo de trabajo GIT **06**

---

Funcionalidad esperada **07**

---

Entregables **08**

---

Notas finales **09**

---

Funcionalidades Opcionales  
(extra) **10**

---

# Índice de contenidos

# Contexto

Imagina que trabajas como desarrollador en un pequeño hotel de ciudad que necesita digitalizar su gestión. Actualmente todo se lleva en hojas de cálculo y quieren una aplicación web que permita gestionar las habitaciones del hotel, los huéspedes registrados, las reservas y las incidencias de mantenimiento. La aplicación deberá permitir a los empleados del hotel (supervisores y recepcionistas) identificarse y, dependiendo de su perfil, realizar distintas operaciones.

El proyecto se desarrollará en pareja, de forma colaborativa, usando GitHub, con ramas independientes para cada miembro del equipo, integración mediante Pull Requests y resolución de conflictos de merge.

El repositorio se creará dentro de la organización de GitHub del curso, donde el tutor tendrá visibilidad de todos los proyectos. Si después queréis tener el proyecto en vuestro GitHub personal, podéis hacerlo una vez finalizado y evaluado.

# Base de datos

## BBDD: hoteldb (MySQL 8.0)

Vuestra aplicación necesita una base de datos. Debéis crearla vosotros mismos en MySQL Workbench. La base de datos se llamará **hoteldb** y tendrá las siguientes **cinco tablas**:

| Tabla               | Propósito                                             |
|---------------------|-------------------------------------------------------|
| <b>usuarios</b>     | Empleados del hotel que se autentican en el sistema   |
| <b>habitaciones</b> | Habitaciones del hotel                                |
| <b>huéspedes</b>    | Clientes / huéspedes del hotel                        |
| <b>reservas</b>     | Reservas de habitaciones por huéspedes                |
| <b>incidencias</b>  | Incidencias de mantenimiento asociadas a habitaciones |

### Relaciones entre tablas:

huéspedes (1) -----< (N) reservas (N) >----- (1) habitaciones (1) >-----< (N) incidencias

La tabla **usuarios** es independiente (solo se usa para el login).

### ¿Qué campos tiene cada tabla?

**Vosotros decidís qué campos (columnas) tiene cada tabla.** Tenéis libertad total, pero debéis respetar estas restricciones mínimas:

- **usuarios:** debe permitir el inicio de sesión (usuario y contraseña) y diferenciar entre perfiles (**repcionista** vs **supervisor**).
- **habitaciones:** debe incluir un número de habitación, un tipo (individual, doble, suite...) y **al menos 3 campos más** que decidáis vosotros (precio por noche, etc.).
- **huéspedes:** debe almacenar datos del cliente (nombre, apellidos, etc.). Vosotros decidís los campos.
- **reservas:** debe vincular un huésped con una habitación. Debe incluir **fechas** (entrada y salida) y **al menos 2 campos más** que decidáis.
- **incidencias:** debe representar incidencias de mantenimiento vinculadas a una habitación. Debe incluir un **estado** (abierta, en curso, cerrada...), una **prioridad** (baja, media, alta...), una **descripción** y **al menos 3 campos más** que decidáis (fecha de apertura, etc.).

**Importante:** Los campos que elijáis deben tener sentido y lógica dentro del contexto de un hotel. No metáis campos por meter solo para cumplir el mínimo — pensad en qué información necesitaría realmente un hotel para gestionar cada entidad.

# 02

# Base de datos

## Datos iniciales:

Debéis insertar datos suficientes para probar la aplicación:

- Al menos 2 usuarios (1 recepcionista y 1 supervisor)
- Al menos 8 habitaciones de distintos tipos
- Al menos 5 huéspedes
- Al menos 5 reservas
- Al menos 5 incidencias de distintos estados y prioridades

## Scripts SQL:

Cada alumno trabaja con su propia base de datos local en su máquina. No hay una BBDD compartida. Cada alumno debe:

- Crear la BBDD hoteldb en su MySQL local.
- Crear las tablas que le correspondan.
- Insertar sus datos de prueba.

Tened en cuenta que la aplicación necesita que todas las tablas existan en la BBDD para funcionar correctamente.

El SQL debe quedar recogido en el repositorio Git en la carpeta sql/, en ficheros separados según su contenido:

- Un fichero de setup con el CREATE DATABASE y USE (se crea en la Fase 0, juntos).
- Un fichero por cada alumno con los CREATE TABLE e INSERT INTO de sus tablas.
- Un fichero de reservas con la tabla de integración (cuando trabajéis en pareja).

Además, debéis incluir en el fichero de setup (en un comentario al inicio del fichero) el orden en el que deben ejecutarse los scripts, para que cualquier persona que clone el repositorio pueda reconstruir la BBDD completa sin tener que adivinar qué fichero va primero. Las tablas tienen relaciones entre ellas (claves foráneas), así que el orden importa.

# Organización y arquitectura

## Nombre del proyecto: SpringHotelApp

El proyecto debe llamarse **SpringHotelApp**. La estructura interna de paquetes, la forma de configurar Spring y la organización del código la decidís vosotros.

**Sois vosotros quienes decidís cómo organizar el código.**

A lo largo del curso hemos visto distintos enfoques: arquitectura en capas, diferentes formas de configurar Spring (solo XML, XML mínimo con anotaciones, o clase Java de configuración)... Tenéis **todos los ejercicios realizados durante el curso como referencia**. Os podéis basar en ellos si lo creéis oportuno.

Lo que sí debéis respetar:

- Los **nombres de clases, métodos y variables deben ser descriptivos y significativos**, tal y como hemos practicado en todos los ejercicios del curso. Nada de nombres genéricos o sin sentido.
- **Sin caracteres especiales en el código**: no uséis tildes, ñ ni ningún otro carácter especial en nombres de clases, métodos, variables, paquetes o ficheros de configuración. El código debe usar únicamente caracteres ASCII (HabitacionController, no HabitaciónController; año, no año). La aplicación, no obstante, sí debe permitir introducir y mostrar correctamente datos que los contengan.
- Debe existir una **separación clara de responsabilidades**: el código que accede a la base de datos no debe estar mezclado con el que gestiona las peticiones HTTP, y las vistas no deben contener lógica de negocio.
- Debéis usar **interfaces** para las operaciones de acceso a datos, con su correspondiente implementación, tal como hemos visto en el curso.
- Debéis crear vuestras propias **excepciones personalizadas** con nombres descriptivos según su propósito (errores de base de datos, credenciales incorrectas, entidad no encontrada, etc.). Estas excepciones deben usarse en el acceso a datos y en los controladores.
- Es **obligatorio** usar un fichero `.properties` para la configuración de la conexión a la base de datos, referenciado desde la configuración de Spring.



# DIVISIÓN DE TAREAS

El proyecto se divide en **dos partes independientes** que se desarrollan en paralelo, más una **fase de integración** conjunta al final.

## Estudiante A — “Habitaciones + Incidencias”

El Estudiante A es responsable de **todo lo relacionado con la gestión de habitaciones y de incidencias de mantenimiento**:

### Habitaciones

- La entidad Java que representa una habitación, con el mapeo ORM correspondiente mediante anotaciones (@Entity, @Table, @Id, etc.).
- El acceso a datos: una interfaz y su implementación con todas las **operaciones CRUD necesarias**. Los nombres de los métodos los decidís vosotros, pero deben ser descriptivos y coherentes con lo visto en el curso.
- El controlador Spring MVC que gestiona las peticiones HTTP relacionadas con habitaciones.
- Las vistas JSP: Habitaciones.jsp, DetalleHabitacion.jsp, FormularioHabitacion.jsp (unificado para alta y modificación). Deben usar JSTL y Form MVC Tags.
- Su script SQL con las sentencias CREATE TABLE e INSERT INTO para la tabla habitaciones.

### Incidencias

- La entidad Java que representa una incidencia de mantenimiento, con el mapeo ORM correspondiente mediante anotaciones y la relación adecuada hacia la entidad de habitación.
- El acceso a datos de incidencias: una interfaz y su implementación con las **operaciones CRUD necesarias** y una **consulta por habitación** que devuelva las incidencias asociadas a una habitación concreta.
- El controlador Spring MVC que gestiona las peticiones HTTP relacionadas con incidencias.
- Las vistas JSP: Incidencias.jsp, DetalleIncidencia.jsp, FormularioIncidencia.jsp (unificado para alta y modificación). Deben usar JSTL y Form MVC Tags.
- Su script SQL con las sentencias CREATE TABLE e INSERT INTO para la tabla incidencias.

# DIVISIÓN DE TAREAS

## Estudiante B — “Huéspedes + Login”

El Estudiante B es responsable de **todo lo relacionado con el login y la gestión de huéspedes**:

- Las entidades Java para usuarios y huéspedes, con sus correspondientes anotaciones.
- El acceso a datos para usuarios: una interfaz y su implementación con la operación de **consulta por credenciales** (para el login). Los nombres los decidís vosotros.
- El acceso a datos para huéspedes: una interfaz y su implementación con todas las **operaciones CRUD necesarias**. Mismos criterios de nomenclatura que el Estudiante A.
- El controlador para el login y cerrar sesión.
- El controlador CRUD de huéspedes.
- Las vistas JSP: Login.jsp, Principal.jsp, Huespedes.jsp, DetalleHuesped.jsp, FormularioHuesped.jsp (unificado para alta y modificacion). Deben usar JSTL y Form MVC Tags.
- Su script SQL con las sentencias CREATE TABLE e INSERT INTO para las tablas usuarios y huespedes.



# FASE DE INTEGRACIÓN — RESERVAS (AMBOS)

Una vez terminadas las partes individuales, los dos estudiantes abordan conjuntamente la funcionalidad de **reservas**. Esta fase está dividida en dos bloques: unos componentes comunes que desarrolla uno de los dos y un reparto intencionado del controlador y las vistas entre ambos.

## Componentes comunes

Uno de los dos alumnos (el que empiece primero) desarrolla los componentes que ambos necesitarán después:

La entidad Java Reserva con las relaciones adecuadas hacia la entidad de habitación y hacia la entidad de huésped, mediante anotaciones.

La interfaz de acceso a datos de reservas y su implementación con las **operaciones CRUD necesarias**.

El script SQL con la tabla reservas y sus datos de prueba.

Una vez estos componentes estén disponibles, ambos alumnos los utilizarán en el siguiente bloque.

## Controlador y vistas divididos

El **controlador de reservas se divide a propósito entre los dos alumnos**, de forma que ambos trabajan sobre el mismo fichero ReservaController.java pero con métodos distintos:

**Estudiante A:** implementa los métodos de consulta (listado, detalle) y eliminación, más las vistas JSP Reservas.jsp y DetalleReserva.jsp.

**Estudiante B:** implementa los métodos de alta y modificación, más la vista JSP FormularioReserva.jsp (unificado para alta y modificación).

Además, **ambos alumnos añaden métodos a un controlador de navegación general** (cada uno los correspondientes a su parte).

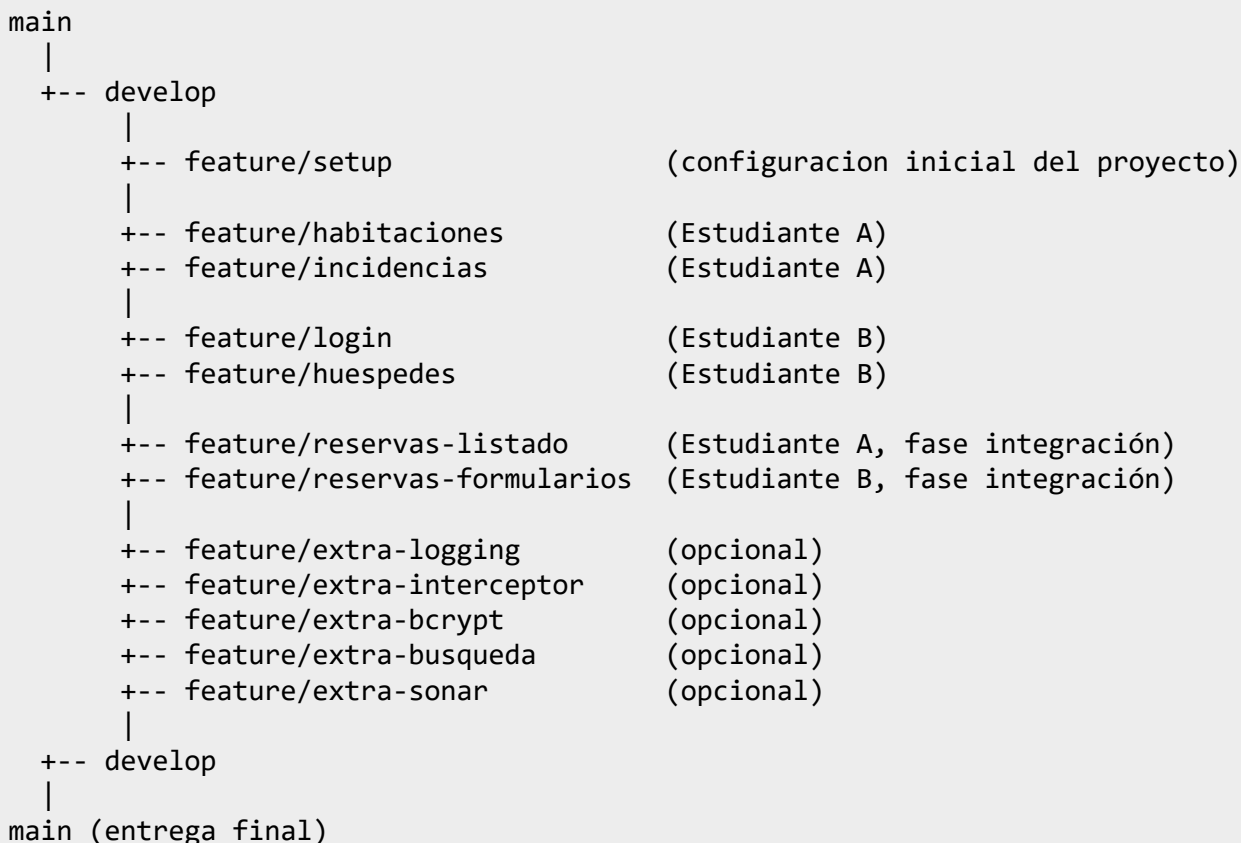
Esta división intencionada del controlador de reservas y del controlador de navegación forma parte del ejercicio: está pensada para que en la integración final ambos alumnos trabajen sobre los mismos ficheros y deban combinar sus cambios en una única versión.

# FLUJO DE TRABAJO GIT

## Repositorio GitHub

Aceptad la invitación de **GitHub** que os proporcionará el tutor. Se creará automáticamente el repositorio del equipo dentro de la organización del curso.

## Estructura de ramas



**IMPORTANTE:** Las ramas **NO se borran** después de mergear. Se mantienen para trazabilidad.

# FLUJO DE TRABAJO GIT

## FASE 0 — Setup inicial (JUNTOS)

Esta fase se hace en conjunto. Cuanto más acordéis al principio, más independientes seréis después.

1. Ambos clonan el repositorio en sus máquinas.
2. Uno de los dos crea la rama develop desde main y la sube al remoto.
3. Ambos crean la rama feature/setup desde develop.
4. **Juntos**, crean el proyecto base:
  - El proyecto con la estructura de carpetas que consideréis adecuada.
  - Los ficheros de configuración necesarios para que Spring MVC + Hibernate funcionen.
  - El fichero .properties con los datos de conexión a MySQL (acordad las mismas credenciales para no estar cambiando el fichero constantemente).
  - Una página de inicio básica.
  - La carpeta sql/ con el fichero de setup de la BBDD.
5. **Juntos**, diseñad la base de datos: acordad los campos de las 5 tablas, sus tipos y sus relaciones. Esto es importante porque ambos necesitaréis conocer la estructura completa al desarrollar las reservas e incidencias.
6. **Juntos**, cread las **excepciones personalizadas** que usaréis en el proyecto. Los nombres los decidís vosotros.

# FLUJO DE TRABAJO GIT

## 7. Juntos, acordad:

- Cómo vais a organizar el código (arquitectura, paquetes, etc.)
- El tipo de configuración de Spring que vais a usar (todos los enfoques vistos en el curso son válidos, pero ambos debéis usar el mismo)
- El reparto de tareas
- Las **convenciones de nomenclatura** que vais a seguir en todo el proyecto. No basta con que cada uno use nombres descriptivos por su cuenta: los nombres deben ser coherentes entre sí. Si uno decide nombrar los métodos de consulta con el verbo obtener, el otro no puede usar listar o mostrar para lo mismo. Si uno llama alta a la operación de creación, el otro no puede llamarla crear o nuevo. El proyecto lo desarrolláis dos personas pero debe leerse como si lo hubiera escrito una sola. Ejemplos de lo que hay que acordar:
  - ¿Cómo se nombran los métodos de lectura? (obtenerHabitaciones / obtenerHuespedes, no listarHabitaciones / mostrarHuespedes)
  - ¿Cómo se nombran los métodos de creación, modificación y eliminación? (alta, actualiza, elimina — o los verbos que elijáis, pero los mismos para todos)
  - ¿Cómo se nombran los controladores, las vistas y las rutas URL? (mismo patrón para habitaciones y para huéspedes)

8. Hacéis commits y push a feature/setup.

9. Creáis un **Pull Request** de feature/setup a develop. Ambos revisáis. Mergeáis. **NO boráis la rama.**

# FLUJO DE TRABAJO GIT

## FASE 1 — Desarrollo paralelo (CADA UNO EN SU RAMA)

Cada uno trabaja en sus ramas de forma independiente. El Estudiante A crea primero feature/habitaciones y, una vez mergeada, crea feature/incidencias. El Estudiante B hace lo mismo con feature/login y después feature/huespedes.

Cada alumno desarrolla su parte completa: base de datos, entidades, acceso a datos, controladores y vistas. Podéis basaros en los ejercicios del curso como referencia.

- Haced **commits regulares** con mensajes descriptivos.
- Pushed a vuestra rama regularmente.

## FASE 2 — Integración y resolución de conflictos

### El primero que termine:

1. Crea un **Pull Request** de su rama a develop.
2. El compañero revisa el PR y deja un comentario de aprobación (por ejemplo, “OK, revisado”). En entornos profesionales, los PR siempre los revisa otra persona antes de mergear — aquí se trata de familiarizarse con esa dinámica.
3. Se mergea. **NO se borra la rama.**

### El segundo (el que mergea después):

1. Actualiza su rama con los cambios de develop.
2. **Aquí aparecen los CONFLICTOS.** Git marcará conflictos en los ficheros que ambos habéis modificado (el fichero de configuración de Spring, index.jsp, el CSS si lo usáis, el menú de navegación, los ficheros SQL, web.xml, etc.).
3. **Resolvéis los conflictos uno por uno**, idealmente con el compañero presente para decidir juntos cómo queda cada fichero.
4. Hacéis commit de la resolución.
5. Creáis PR de vuestra rama a develop. El compañero revisa que la resolución es correcta.
6. Se mergea. **NO se borra la rama.**

# 06

# FLUJO DE TRABAJO GIT

## FASE 3 — Reservas e integración final

Seguid los pasos indicados en la sección “**Fase de Integración — Reservas**” de este documento:

1. Desde develop (ya con ambas ramas mergeadas), cada alumno crea su propia rama: feature/reservas-listado (Estudiante A) y feature/reservas-formularios (Estudiante B).
2. Uno de los dos crea los componentes comunes (entidad, DAO, SQL) en su rama, hace PR a develop y se mergea. El otro actualiza su rama.
3. Cada uno implementa sus métodos del controlador de reservas, sus JSPs y sus métodos del controlador de navegación.
4. El primero que termine: PR a develop, se mergea.
5. El segundo actualiza su rama con develop → **conflictos en el controlador de reservas y en el de navegación** → los resolvéis fusionando los métodos de ambos.
6. PR a develop. Ambos revisáis.
7. Unificáis la navegación, implementáis los roles, probáis los flujos completos.
8. PR final de develop a main.
9. **NINGUNA rama se borra.**

# FUNCIONALIDAD ESPERADA

Una vez terminada, la aplicación debe funcionar así:

## Login

- Arranca con la página de **Login**.
- Si usuario/contraseña vacíos o incorrectos: mensaje de error, sigue en Login.
- Si correctos: se crea la sesión y se accede a la página **Principal**.

## Página Principal

- Muestra los datos del empleado logado.
- Menú de navegación: **Habitaciones | Huéspedes | Reservas | Incidencias | Cerrar sesión**
- Cerrar sesión invalida la sesión y vuelve al Login.

## Habitaciones

- **Listado** con todas las habitaciones.
- Según el perfil del usuario, se muestran las opciones de **Detalle, Modificar, Eliminar y Agregar**.
- **Detalle**: toda la información de la habitación.
- **Alta**: formulario vacío para crear una nueva habitación.
- **Modificación**: formulario precargado con los datos actuales.
- **Eliminación**: elimina y vuelve al listado.

## Huéspedes

- Misma estructura que Habitaciones, pero para huéspedes.

## Reservas

- **Listado** con reservas (huésped, habitación, fechas).
- **Detalle** de una reserva con toda su información.
- **Alta**: formulario con selección de huésped y habitación existentes.
- **Modificación y Eliminación**.



# FUNCIONALIDAD ESPERADA

## Incidencias

- **Listado** de todas las incidencias con su estado y prioridad.
- **Detalle** de una incidencia con toda su información.
- **Alta**: formulario para crear una nueva incidencia seleccionando la habitación asociada.
- **Modificación** del estado, prioridad y demás campos.
- **Eliminación**.
- Desde el **detalle de una habitación**, enlace para ver las incidencias asociadas a esa habitación.

## Roles

- **Recepcionista**: acceso a todas las operaciones (consultar, crear, modificar, eliminar).
- **Supervisor**: acceso solo a consulta/detalle.

# ENTREGABLES

**Lo que se entrega al tutor es el repositorio de GitHub.** El tutor evaluará el estado de la rama main tras el último merge. Aseguraos de que todo está correctamente mergeado en main antes de dar el proyecto por entregado.

## 1. Repositorio en GitHub

El repositorio del equipo con:

- Ramas visibles: main, develop, feature/setup, feature/habitaciones, feature/incidencias, feature/login, feature/huespedes, feature/reservas-listado, feature/reservas-formularios (y las ramas feature/extra-\* si se realizaron funcionalidades opcionales)
- Al menos **4 Pull Requests cerrados** con algún comentario de review
- Historial de commits coherente
- Al menos **1 merge con conflictos resueltos** visible en el historial

## 2. Scripts SQL (carpeta sql/)

Los scripts SQL de cada alumno y el de integración, organizados de forma que puedan ejecutarse en orden para reconstruir la BBDD completa. El Estudiante A incluye los scripts de las tablas habitaciones e incidencias. El Estudiante B incluye los scripts de las tablas usuarios y huespedes.

## 3. Aplicación funcionando

Se verificará en la defensa:

- Login con dos perfiles (recepcionista y supervisor)
- CRUD completo de habitaciones
- CRUD completo de huéspedes
- CRUD completo de incidencias
- CRUD de reservas (si trabajáis en pareja)
- Roles: recepcionista ve todo, supervisor solo consulta
- Navegación coherente entre páginas

# 08

# ENTREGABLES

## 4. README.md

En la raíz del repositorio, un README.md con las siguientes secciones:

- **Descripción del proyecto:** qué es la aplicación y qué permite hacer (1-2 párrafos).
- **Requisitos previos:** qué necesita alguien para ejecutar el proyecto (versión de Java, MySQL, servidor Tomcat, etc.).
- **Instalación y despliegue:** pasos para clonar el repositorio, crear la base de datos (en qué orden ejecutar los scripts SQL) y desplegar la aplicación en el servidor.
- **Tecnologías utilizadas:** listar las tecnologías y librerías principales (Spring MVC, Hibernate, MySQL, JSTL, Log4j, BCrypt... las que hayáis usado).
- **Estructura del proyecto:** breve explicación de cómo habéis organizado los paquetes y la arquitectura.
- **Reparto de tareas:** qué ha desarrollado cada alumno.
- **Conflictos de merge:** qué conflictos encontrasteis al integrar las ramas y cómo los resolvisteis.
- **Funcionalidades extra** (si las habéis implementado): cuáles y una breve descripción de cada una. Si habéis hecho el Extra 5 (SonarQube), incluir la captura de pantalla del dashboard con las métricas.
- **Credenciales de prueba:** usuarios y contraseñas para poder probar la aplicación (por ejemplo: recepcionista/recep123, supervisor/super123).

# NOTAS FINALES

- Tenéis **todos los ejercicios realizados a lo largo del curso** a vuestra disposición como referencia. Si en algún momento no sabéis cómo afrontar algo (configurar Spring, mapear una entidad, crear un controlador, trabajar con sesiones...), revisad los ejercicios anteriores: el proyecto sigue los mismos patrones que ya habéis practicado.
- Las vistas deben **funcionar correctamente** y mostrar los datos. No es necesario que sean elaboradas visualmente, pero si lo hacéis se tendrá en cuenta. Recordad que esto es un curso de backend Java.
- Elegid bien la configuración de Spring antes de empezar y aseguraos de que ambos usáis el mismo enfoque. Cambiarlo a mitad del desarrollo es costoso.
- Cuanto más consensuéis en la Fase 0 (estructura del proyecto, campos de la BBDD, organización del código), más fluido será el desarrollo paralelo posterior.

# FUNCIONALIDADES OPCIONALES (EXTRA)

**IMPORTANTE:** Estas funcionalidades son **completamente opcionales** y solo deben abordarse **una vez que todo lo anterior esté terminado y funcionando**. Están pensadas para parejas que terminen antes de tiempo y quieran mejorar su proyecto con prácticas que se usan en entornos profesionales.

Cada funcionalidad es independiente. Podéis hacer una, varias o todas, en el orden que se presenta. Cread una rama desde develop para cada extra (por ejemplo, feature/extra-logging, feature/extra-interceptor, etc.).

## Extra 1 — Logging con Log4j (feature/extra-logging)

**Objetivo:** Sustituir todos los `System.out.println` del proyecto por un sistema de logging profesional.

**Por qué:** En entornos profesionales no se usa `System.out.println`. Log4j permite controlar qué mensajes se muestran según su nivel (DEBUG, INFO, WARN, ERROR), enviar los logs a fichero además de a consola, y activar o desactivar mensajes sin tocar el código.

### Qué hacer:

1. Añadir la librería Log4j al proyecto.
2. Crear un fichero `log4j.properties` con una configuración que envíe los logs tanto a consola como a un fichero `hotel.log`.
3. En cada clase que tenga `System.out.println`, crear un logger estático: `private static final Logger logger = Logger.getLogger(NombreClase.class);`
4. Sustituir cada `System.out.println` por el nivel de log adecuado: `logger.info()` para flujo normal, `logger.error()` para excepciones, `logger.debug()` para datos de depuración.
5. Verificar que no queda ningún `System.out.println` en el proyecto y que los logs aparecen correctamente.

# FUNCIONALIDADES OPCIONALES (EXTRA)

## Extra 2 — Interceptor de seguridad (feature/extra-interceptor)

**Objetivo:** Proteger las rutas de la aplicación para que nadie pueda acceder sin estar logado.

**Por qué:** Ahora mismo, si alguien escribe directamente en el navegador la URL de habitaciones o huéspedes, puede entrar sin hacer login. El sistema de login no tiene efecto real si las rutas no están protegidas. Un HandlerInterceptor comprueba en cada petición si hay sesión activa y redirige al login si no la hay.

### Qué hacer:

1. Crear una clase que implemente HandlerInterceptor (en un paquete adecuado, por ejemplo interceptores).
2. En el método preHandle, comprobar si la HttpSession contiene el atributo del usuario logado.
3. Si no hay sesión activa: redirigir a la página de login y devolver false. Si hay sesión: devolver true.
4. Registrar el interceptor en la configuración de Spring, aplicándolo a todas las rutas y excluyendo /login y los recursos estáticos.
5. Probar: sin estar logado, intentar acceder directamente a cualquier URL → debe redirigir al login. Después de hacer login, todo debe funcionar con normalidad.

# FUNCIONALIDADES OPCIONALES (EXTRA)

## Extra 3 — Cifrado de contraseñas con BCrypt (feature/extra-bcrypt)

**Objetivo:** Dejar de almacenar las contraseñas en texto plano en la base de datos.

**Por qué:** Guardar contraseñas sin cifrar es una vulnerabilidad crítica. Si alguien accede a la base de datos, tiene todas las contraseñas. Con BCrypt, se almacena un hash irreversible y se compara de forma segura al hacer login.

### Qué hacer:

1. Añadir la librería jBCrypt al proyecto.
2. Crear una clase utilidad con dos métodos estáticos: uno para generar el hash de una contraseña y otro para comparar una contraseña en texto plano contra un hash.
3. Actualizar los INSERT INTO de la tabla usuarios: en lugar de la contraseña en texto plano, insertar el hash BCrypt correspondiente (podéis generarlo con un main temporal).
4. Modificar la lógica de login en el DAO de usuarios: ya no se busca por contraseña en la consulta. Se busca solo por nombre de usuario, se recupera el hash almacenado y se compara usando el método de verificación de BCrypt.
5. Verificar en MySQL Workbench que la columna de contraseña muestra hashes y no texto plano. Probar que el login sigue funcionando correctamente.

# FUNCIONALIDADES OPCIONALES (EXTRA)

## Extra 4 — Búsqueda y filtrado en listados (feature/extra-busqueda)

**Objetivo:** Añadir funcionalidad de búsqueda en las páginas de listado para localizar registros sin recorrer toda la tabla visualmente.

**Por qué:** Cualquier aplicación de gestión real tiene buscadores. Esta funcionalidad demuestra manejo de HQL parametrizado y procesamiento de parámetros opcionales.

### Qué hacer:

1. Decidir qué criterios de búsqueda implementar (ejemplos: habitaciones por tipo o rango de precio, huéspedes por nombre o documento de identidad, reservas por fecha).
2. Añadir un método en la interfaz del DAO e implementarlo con una consulta HQL parametrizada que soporte filtros opcionales (si un filtro viene vacío, no se aplica).
3. Añadir un método en el controlador que reciba los parámetros de búsqueda como `@RequestParam(required = false)`.
4. Modificar la JSP del listado: añadir un formulario de búsqueda encima de la tabla (método GET) con los campos de filtro y un botón de buscar.
5. Si no se aplica ningún filtro, deben mostrarse todos los resultados (comportamiento por defecto). Si la búsqueda no devuelve resultados, mostrar un mensaje informativo.
6. Implementar la búsqueda en al menos **dos listados** distintos.



# FUNCIONALIDADES OPCIONALES (EXTRA)

## Extra 5 — Verificación de calidad de código con SonarQube (feature/extra-sonar)

**Objetivo:** Analizar el código del proyecto con un servidor SonarQube local y cumplir unos umbrales mínimos de calidad.

**Por qué:** En entornos profesionales, el código no solo debe funcionar: debe cumplir estándares de calidad medibles. SonarQube es la herramienta más utilizada en la industria para análisis estático de código. Detecta bugs potenciales, vulnerabilidades de seguridad, code smells y código duplicado. Aprender a trabajar con estas métricas os prepara para las exigencias reales de cualquier proyecto empresarial.

### Qué hacer:

1. Instalar y arrancar un servidor **SonarQube Community Edition** en local (es gratuito). Tomad como guía el vídeo de explicación de SonarQube visto en la tutoría.
2. Ejecutar el análisis del proyecto y revisar los resultados en el dashboard de SonarQube.
3. Corregir los issues detectados hasta que el proyecto cumpla **todas** las siguientes métricas:

| Métrica                  | Umbral requerido |
|--------------------------|------------------|
| <b>Security</b>          | A                |
| <b>Reliability</b>       | A                |
| <b>Maintainability</b>   | A                |
| <b>Hotspots Reviewed</b> | A                |
| <b>Duplications</b>      | < 3%             |

4. Una vez alcanzadas las métricas, hacer una captura de pantalla del dashboard de SonarQube mostrando los resultados y añadirla al README.md del proyecto.

**Gracias**